KIT/BSQBD/KSQBD - noSQL systémy a Big Data

Navigace
- Titulní stránka
  - Nástěnka
  - Moje kurzy
    - JC/UPCUH (LS, 2025/2026)
    - JC/UPCUH (ZSiLS, 2025/2026)_2
    - JC/UPCUH (ZSiLS, 2025/2026)
    - KIT/BUSE (LS, 2025/2026)
    - KST/BPOG1 (LS, 2025/2026)
    - KIT/BBIS (LS, 2025/2026)
    - KIT/BMTA (ZS, 2025/2026)
    - KST/BCSH2 (ZS, 2025/2026)
    - KST/BMOSI (ZS, 2025/2026)
    - KIT/BSQBD/KSQBD (ZS, 2025/2026)
      - Účastníci
      - Známky
      - Zápočet, Zkouška
        - Základní samokontrola semestrální práce před odevz...
    - KIT/BDAS2 (ZS, 2025/2026)
    - FES/BOZP poučení (ZS, 2025/2026)
    - KST/BCSH1 (LS, 2024/2025)
    - KIT/BRPP2 (LS, 2024/2025)
    - 2025 – OS
    - KAM/BTEGR (LS, 2024/2025)
    - KST/BDATS (ZS, 2024/2025)
    - KST/BOOP (LS, 2024/2025)
    - KST/BPOG1 (LS, 2024/2025)
    - KIT/BSPWE (LS, 2024/2025)
    - KIT/BOPE2 (ZS, 2024/2025)_2
    - KIT/BOPE2 (ZS, 2024/2025)
    - 2025 – PS1
    - KAM/BAMAT (ZS, 2024/2025)_KOMPAS
    - KAM/BAMAT (ZS, 2024/2025)
    - KIT/BZALG (ZS, 2024/2025)
    - 2025 – IZ (ZS)
    - JC/IPAIH (LS, 2024/2025)
    - JC/IPAIG (ZS, 2024/2025)
    - JC/IPAIF (LS, 2023/2024)
    - Více...

Základní samokontrola semestrální práce před odevzdáním

OBECNÉ

- Zobrazení logů konkrétní služby
  `docker compose logs <service_name>`
  Očekávání: v logu nejsou chybová hlášení při startu (např. authentication failed, bind failed, out of memory) a u databází jsou uvedeny informace o úspěšném startu clusteru.

- Restart konkrétní služby (simulace výpadku)
  `docker compose restart <service_name>`
  Očekávání: dojde ke krátkodobému výpadku, ale cluster se po chvíli stabilizuje a klientské dotazy jsou opět obsluhovány.

- Stop/start pro simulaci výpadku uzlu
  `docker compose stop <service_name>`
  `docker compose start <service_name>`
  Očekávání: při výpadku se projeví degradace (např. horší dostupnost, žlutý stav clusteru, snížený replikační faktor), po opětovném spuštění služby se stav zlepší.

REDIS

- Připojení do Redis CLI (například do jednoho z uzlů)
  `redis-cli -h <hostname> -p <port>`
  Očekávání: zobrazení ve tvaru 127.0.0.1:6379> (nebo jiný host/port dle konfigurace).
  Základní ping
  `PING`
  Očekávání: odpověď `PONG`, což potvrzuje, že uzel běží.

- Informace o clusteru
  `redis-cli -h <host> -p <port> CLUSTER INFO`
  Očekávání: klíč `cluster_state:ok`, správné počty slotů, žádné stavy `fail`.

- Seznam uzlů
  `redis-cli -h <host> -p <port> CLUSTER NODES`
  Očekávání: všechny uzly typu `master`/`slave` mají status `connected`, nevyskytují se stavy `handshake` ani `fail`.

- Kontrola rozložení slotů (distribuce dat)
  `redis-cli -h <host> -p <port> CLUSTER SLOTS`
  Očekávání: sloty 0–16383 jsou pokryty, u každého mastera jsou uvedeny případné repliky.

- Kontrola počtu klíčů po importu dat
  `DBSIZE`
  Očekávání: počet klíčů odpovídá očekávané velikosti nahraného datasetu.

- Kontrola rolí na konkrétním uzlu
  `INFO replication`
  Očekávání: master uvádí `role:master`, replika uvádí `role:slave` a `master_link_status:up`.

- Simulace výpadku master uzlu
  `docker compose stop redis-node-1` (podle názvu služby)
  Následně:
  `redis-cli -h <jiný_host> -p <port> CLUSTER INFO`
  Očekávání: cluster zůstává ve stavu `cluster_state:ok` (při správně nastavených replikách a quorum) a jiný uzel přebírá roli mastera pro příslušné sloty.

- Návrat uzlu do provozu
  `docker compose start redis-node-1`
  `CLUSTER NODES`
  Očekávání: uzel je znovu součástí clusteru, replikace je stabilizována.

MONGODB

- Základní health check
  - Ping databáze (přes mongosh)
    `mongosh --eval 'db.runCommand("ping")'`
    Očekávání: dokument obsahující `ok: 1`.
  - Info o buildu (verze)
  - `mongosh --eval 'db.runCommand({ buildInfo: 1 })'`
    Očekávání: pole `version` odpovídá použité verzi (>= 8.0.x dle zadání).

- Replika set / cluster
  - Status repliky (na primáru)
  - `mongosh --eval 'rs.status()'`
  - Očekávání:
    - `set` – jméno replika setu
    - node se `stateStr: "PRIMARY"`
    - další s `stateStr: "SECONDARY"`
    - `ok: 1`
  - Konfigurace repliky
  - `mongosh --eval 'rs.conf()'`
    Očekávání: počet členů = počet nodů, replikationFactor odpovídá zadání.
- Sharding
  - Stav shardu
    `mongosh --eval 'sh.status()'`
    Očekávání:
    - seznam shardů (min. 3)
    - rozdělení chunků podle klíče
    - počet chunků na shardech odpovídá popisu

- Statistika a distribuce dat
  - Stav databáze
    - `mongosh --eval 'db.stats()'`
    - Očekávání: `collections`, `objects`, `dataSize`, `storageSize` odpovídají datům.
  - Statistika kolekce
    - `mongosh --eval 'db.yourCollection.stats()'`
    - Očekávání: `count` = počet dokumentů, `sharded: true` pokud je kolekce shardovaná, rozdělení shardů v poli `shards`.
  - Indexy v kolekci
    - `mongosh --eval 'db.yourCollection.getIndexes()'`
    - Očekávání: primární a sekundární indexy, které jsou popsány v dokumentaci
- Zabezpečení
  - Přihlášení s uživatelem
    `mongosh -u <user> -p <password> --authenticationDatabase admin`
    Očekávání: přihlášení úspěšné.

  - Kontrola uživatelů a rolí
    `mongosh --eval 'db.getUsers()'` (v odpovídající DB)
    Očekávání: definovaní uživatelé s rolemi (`readWrite`, `dbAdmin`, apod.).

- Kontrola keyfile a interní autentizace
  - Ověření, že je keyfile uveden v konfiguraci MongoDB.
  - `grep -n "keyFile" /etc/mongod.conf`
    nebo
    `cat /etc/mongod.conf`
    Očekávání: je uvedena položka `security.keyFile:` s cestou ke keyfile, případně je při startu použit parametr `--keyFile`. MongoDB pro interní autentizaci členů replika setu nebo sharded clusteru používá právě `security.keyFile` nebo `--keyFile`.
  - Ověření oprávnění ke keyfile.
  - `ls -l <cesta-ke-keyfile>`
    Očekávání: soubor nemá group ani world permissions; na UNIX systémech je doporučeno použít omezená oprávnění, typicky `chmod 400` nebo `chmod 600`. Keyfile nesmí být ponechán jako nechráněný statický soubor.
  - Ověření, že keyfile není sdílen jako veřejně přístupný statický soubor.
  - `stat <cesta-ke-keyfile>`
    Očekávání: vlastníkem je účet používaný pro běh MongoDB a soubor je chráněn proti neautorizovanému přístupu.
  - Ověření interní autentizace mezi uzly.
    `mongosh --eval 'rs.status()'`
    Očekávání: replika set funguje bez chyb autentizace mezi uzly a členové clusteru se mezi sebou ověřují pomocí keyfile.

Apache Cassandra

- Základní stav clusteru
  - `nodetool status`
  - Očekávání: Sloupec `Owns` ukazuje rozdělení tokenů/dat mezi uzly. U všech uzlů je stav `UN` (Up/Normal).
  - `nodetool describecluster`
  - Očekávání: je vypsán název clusteru, snitch, partitioner a schema verze. Pro kontrolu partitioningu je důležité, že partitioner odpovídá očekávané konfiguraci, typicky `Murmur3Partitioner`.
  - `DESCRIBE CLUSTER` v `cqlsh`
    Očekávání: je uveden název clusteru a partitioner, což potvrzuje, jakým způsobem je data rozdělována.
- Informace o konkrétním uzlu
  - `nodetool info`
  - Očekávání: zobrazení údajů o uptime, zatížení (load), počtu tokenů, bez významných chyb v hlášeních.
- Replikační faktor a keyspaces
  - Popis keyspace (replikační faktor) v cqlsh
  - `DESCRIBE KEYSPACE your_keyspace;`
  - Očekávání: v definici keyspace je uveden `replication = {...}`, kde hodnota replikace odpovídá požadovanému replikačnímu faktoru.
- Distribuce dat a latence
  - Statistika tabulek
    - `nodetool tablestats your_keyspace`
    - Očekávání: výpis obsahuje hodnoty `Space used`, `Number of keys`, `Read Latency`, `Write Latency` a další metriky, které odpovídají očekávanému množství dat a provozu.
  - Informace o ring (pokud je k dispozici)
    - `nodetool ring` (u některých verzí)
    - Očekávání: rovnoměrné rozložení tokenů mezi uzly.
- Kontrola partition key a tokenů
  - `SELECT token(partition_key) FROM keyspace.table WHERE partition_key = 'hodnota';`
  - Očekávání: je vrácen token konkrétní partition key. Stejná partition key musí vracet stejný token, což potvrzuje správné mapování do stejné partition. Použití `token()` je běžný způsob, jak ověřit, kam se daný partition přiřazuje.
  - `SELECT token(partition_key), * FROM keyspace.table WHERE partition_key = 'hodnota';`
  - Očekávání: je vrácen konkrétní záznam spolu s tokenem, takže je možné ověřit, že záznam patří do očekávané partition.
  - `SELECT token(partition_key) FROM keyspace.table LIMIT 10;`
  - Očekávání: pro různé partition key se zobrazují různé tokeny, což ukazuje, že data jsou rozdělována podle partitioningu.
- Ověření dostupnosti dat v partition
  - `SELECT * FROM keyspace.table WHERE partition_key = 'hodnota';`
  - Očekávání: jsou vráceny všechny řádky z dané partition. V Cassandře je přístup k datům navržen primárně přes partition key, takže tento dotaz potvrzuje, že partition je funkční a data jsou čitelná.
  - `SELECT * FROM keyspace.table WHERE partition_key = 'hodnota' AND clustering_col > X;`
  - Očekávání: jsou vráceny pouze řádky v rámci konkrétní partition, které splňují clustering podmínku. Tím je možné ověřit nejen partitioning, ale i správné chování clustering sloupců.
  - `SELECT COUNT(*) FROM keyspace.table;`
  - Očekávání: vrácen je celkový počet řádků. Tento příkaz je vhodný spíše na kontrolu dat po importu než na běžné používání, protože může být nákladný.
- Distribuce dat a rozložení na uzlech
  - `nodetool tablestats keyspace`
  - Očekávání: jsou uvedeny metriky jako `Number of keys`, `Space used`, `Read Latency` a `Write Latency`. `Number of keys` pomáhá ověřit, kolik partition nebo klíčů je v tabulce skutečně uloženo.
  - `nodetool cfstats keyspace.table`
  - Očekávání: výpis obsahuje detailní statistiky pro konkrétní tabulku, tedy i přibližný počet partition a rozložení dat. Tento příkaz je vhodný pro dokumentaci distribuce dat.
  - `SELECT * FROM system.peers;`
  - Očekávání: jsou uvedeny ostatní uzly clusteru a jejich token ranges, což pomáhá ověřit, jak jsou partitiony rozloženy napříč clusterem.

ELK

Kontrola je obvykle prováděna v Kibana Dev Tools nebo přes HTTP (curl).

- Zdraví clusteru
  `GET _cluster/health`
  Očekávání: v odpovědi je uvedeno `status` = `green` (ideálně) nebo `yellow` (např. nedostupné repliky), `number_of_nodes` je alespoň 3.

- Seznam uzlů
  `GET _cat/nodes?v`
  Očekávání: všechny uzly jsou vypsány, role (`m`, `d`, `i` apod.) odpovídají konfiguraci (master/data/ingest).

- Informace o indexech
  `GET _cat/indices?v`
  Očekávání: seznam indexů s poli `docs.count`, `store.size` a dalšími hodnotami odpovídajícími nahraným datům.

- Shardy
  `GET _cat/shards?v`
  Očekávání: pro každý index je uveden seznam shardů, jejich stav (`STARTED`, případně `RELOCATING`) a uzel, na kterém běží.

- Alokace shardů (velikost na uzlech)
  `GET _cat/allocation?v`
  Očekávání: pro každý uzel jsou uvedeny například `shards`, `disk.indices`, což umožňuje kontrolu distribuce dat.

- Replikace a recovery (replikační zpoždění)
  `GET _cat/recovery?v`
  Očekávání: při stabilním stavu je výpis prázdný nebo krátký, během rebalancování či obnovy jsou zobrazeny probíhající operace a jejich průběh.

- Statistika indexu (počty dokumentů)
  `GET index_name/_stats/docs`
  Očekávání: `docs.count` odpovídá očekávanému počtu dokumentů v daném indexu.

- Základní vyhledávání
  `GET index_name/_search?size=5`
  Očekávání: v poli `hits.hits` je vráceno 5 dokumentů, což potvrzuje, že index obsahuje data.

- Dotaz s filtrem
  `GET index_name/_search`

  ```
  json
  {
    "query": {
      "bool": {
        "filter": [
          { "term": { "status": 200 } }
        ]
      }
    }
  }
  ```

  Očekávání: vrácené dokumenty mají pole `status` s hodnotou 200.

- Dotaz s agregací (terms)
  `GET index_name/_search`

  ```
  json
  {
    "size": 0,
    "aggs": {
      "by_status": {
        "terms": { "field": "status" }
      }
    }
  }
  ```

  Očekávání: v části `aggregations.by_status.buckets` je uveden seznam statusů a počty dokumentů pro jednotlivé hodnoty.

- Základní informace o Logstash nodu
  `curl -XGET 'http://localhost:9600/?pretty'` (port je závislý na konfiguraci)
  Očekávání: JSON s údaji `host`, `version`, `http_address`.

- Statistika uzlu
  `curl -XGET 'http://localhost:9600/_node/stats?pretty'`
  Očekávání: zobrazení metrik pipeline, počtů zpracovaných událostí a dalších statistik bez kritických chyb.

Kibana je obvykle ověřována přes webové rozhraní, lze však použít i jednoduchou kontrolu přes HTTP.

- Kontrola dostupnosti pomocí HTTP
  `curl -I http://localhost:5601`
  Očekávání: HTTP odpověď `200 OK` nebo `302`, což potvrzuje, že Kibana běží.

- Ověření přihlášení v prohlížeči
  Otevření `http://localhost:5601`
  Očekávání: zobrazení přihlašovací obrazovky a po přihlášení dostupnost sekce Discover s index patterns a daty.

Naposledy změněno: čtvrtek, 26. března 2026, 15.54

◄ Důvody vedoucí k zamítnutí práce - binární kritéria
Přejít na...
Šablona ►